

Report for ELearning Application

eIntellectual App

Final Report September 2024

Advance Web Development

By Mayra Torres Noyola

Final Report September 2024	1
Advance Web Development	1
Introduction	4
Aims	4
Objectives	4
App Requirements Specification	4
Project Setup	5
Environment	5
Operating System	5
Virtual Environment	5
Python Version	6
Packages	6
Project Creation	6
Application Structure	7
Configuration	7
Database	8
Data	8
Entity Relation Model	9
Data Normalization	10
First Normal Form	10
Second Normal Form	10
Third Normal Form	11
Django Data Models	12
Migrations	13
Django Admin Setup	13
Data Loading	13
Views and Templates	13
Views	13
Templates	14
URLs Configuration	14
Validating and Securing Data	15
Forms	15
Validations	15
Security	15
Authentication and Permissions	15
Users Accounts	15
Authentication in Templating	16
Authentication and Permissions in Views	16
Authentication and Permissions with Decorators	16

Testing	17
Unit Testing	17
App Requirements Specification Evaluation	17
Conclusion	18
References	19
Appendix	20

Introduction

Aims

The aim of this project is to create an eLearning application where users can access information, as students and teachers, about their courses. The focus of this report is to show the reasoning behind the choices of each step of the development process of the app.

Objectives

The objectives of this report are:

- Describe the requirements of the project
- Define the development requirements for the app
- Setup a Django project and outline its structure
- Create the dataset and narrate the reasons of the chosen entities.
- Show the process for preparing and loading the data into the database
- Describe Django views and HTML templates used in the application
- Delineate the methodology used to validate incoming data
- Explain authentication and permissions processes.
- Test the application performance and describe the tests
- Show the creation of APIs for the application data
- Evaluate if the project requirements were fulfilled
- Reflect on unfulfilled requirements
- Reflect on the possibilities of the application.

App Requirements Specification

The requirements of the project is to create an eLearning app that can provide the following functional interface for the user [“How to Write a Software Requirements Specification”].

- Users
 - Users can register as students or as teachers
 - Users can login and logout
 - Users have a private profile
 - Users can update private profile and change their own password
 - Students and teachers have a public home page to show some of the profile information.
- Teachers
 - Teacher home page show three most recents created course
 - Teachers see a page with a list of created courses
 - Teacher should be able to create a new course
 - Teacher can update their own course by uploading new resources, including files

- Teachers see a list of students enrolled in the teacher's courses
 - Teacher are able to unenroll (remove) students from the teacher's courses
- Students
 - Students home page should show three most recents enrolled courses
 - Students should see a page with a list of enrolled courses
 - Students enroll themselves in a course
- Course
 - Shows a list of courses in the app
 - Display the minimum information of a course if student not enrolled or teacher is not the owner
 - Display course resources with a link to a resource when the student is enrolled or when the teacher is the owner.
 - Presents a form to create a course
 - Courses can be removed or deleted

Project Setup

The following points describe the reasoning for each of the steps on the development process of the project.

Environment

Operating System

The operating system in my computer is Windows 11. Due to the best compatibility with libraries, I installed **Windows Subsystem for Linux 2 (WSL2)** on my computer. WSL2 provides a Linux environment in my Windows machine ["What is Windows Subsystem for Linux | Microsoft Learn"].

Virtual Environment

For the virtual environment, the options were the following

- `Virtualenv` "is a tool to create isolated Python environments"["virtualenv"]
- `Virtualenvwrapper` " is a set of extensions to `Virtualenv`" [Hellmann].
 - These extensions can facilitate the creation of virtual environments in one place.
 - "Switch between environments with single command" [Hellmann]

Due to the two points mentioned above, I use `Virtualenvwrapper` for this project.

```
/eLearningApp$ mkvirtualenv -p /usr/bin/python3 eApp
```

```
/eLearningApp$ workon eApp
```

Python Version

I found that the best python version to work with all packages in the following section was the following

- Python 3.8.10

Packages

Project Creation

To create the project I use the directory **eLearningApp**, it contains the following:

- Project directory: **eIntellectual**.

```
(eApp) (base) mayraosccarr@LAPTOP-NMB3103K:~/eLearningApp$ django-admin startproject eIntellectual
```

In this directory the following apps were added:

```
/eLearningApp/eIntellectual$ python manage.py startapp management
```

```
/eLearningApp/eIntellectual$ python manage.py startapp students
```

```
/eLearningApp/eIntellectual$ python manage.py startapp teachers
```

```
/eLearningApp/eIntellectual$ python manage.py startapp courses
```

```
/eLearningApp/eIntellectual$ python manage.py startapp chat
```

- Management:
 - Create home and base pages
 - Process and pass data used in every page
 - Manage users deletion
 - Students:
 - Registering and creating an account
 - Login and logout as student
 - Public home details
 - Create, edit or change password of private profile
 - Create an enrollment list
 - Teachers:
 - Registering and creating an account
 - Login and logout as teacher
 - Public home details
 - Create, edit or change password of private profile
 - Creating a list of created courses
 - Show students enrolled in the teacher's course
 - Course:
 - Create, update and delete a course
 - Show the course and resources
 - Show a list of courses available
 - Enroll and unenroll student from course
 - Chat
- Files for testing or showing the app

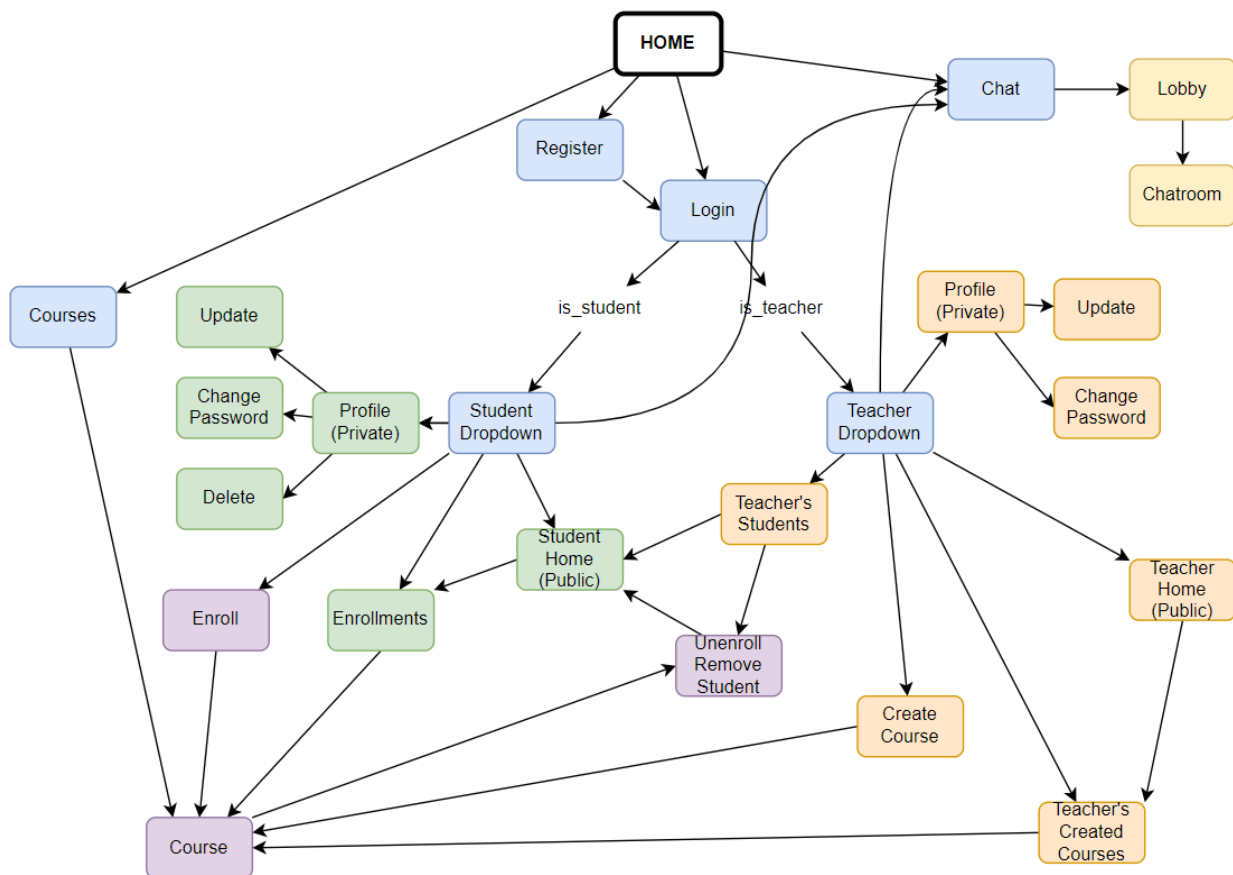
- And requirements.txt file

Application Structure

The following sitemap represents the links in each page, where each page is represented by a node. The following is a visual representation of the hierarchical structure of the website["Site map"].

The colors in this sitemap represent the app that handles each view, with the exception of blue that represents the links in the home page.

Sitemap Navigation Links



Configuration

Before starting to code each part of the application, the project's apps were added to the settings file. And the project's urls were configured depending on the order in which the page needs to be accessed. For example, students and teachers are the main page for their

respective subpages. Management and courses app did not need to be the main page because each one represents stand alone pages.

```
urlpatterns = [
    path('', include('management.urls')),
    path('', include('courses.urls')),
    path('student/', include('students.urls')),
    path('teacher/', include('teachers.urls')),
    path('chat/', include('chat.urls')),
    path("admin/", admin.site.urls),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT) # serve media files
```

For HTML templates for the main page, a directory was created in the project file to be accessed directly from all the apps. Therefore, the path for the directory was updated in the settings file.

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR/"templates"],
    },
]
```

When creating the student and teacher models, the original Django User model needed to be customized to recognize between each type of user. By using as a base the User model, the application would still have a user but with different permissions and profiles for students and for teachers. Therefore the authentication user model needed to be set to the new customized User model.

```
# sets the authentication app
AUTH_USER_MODEL = 'management.User'
```

Database

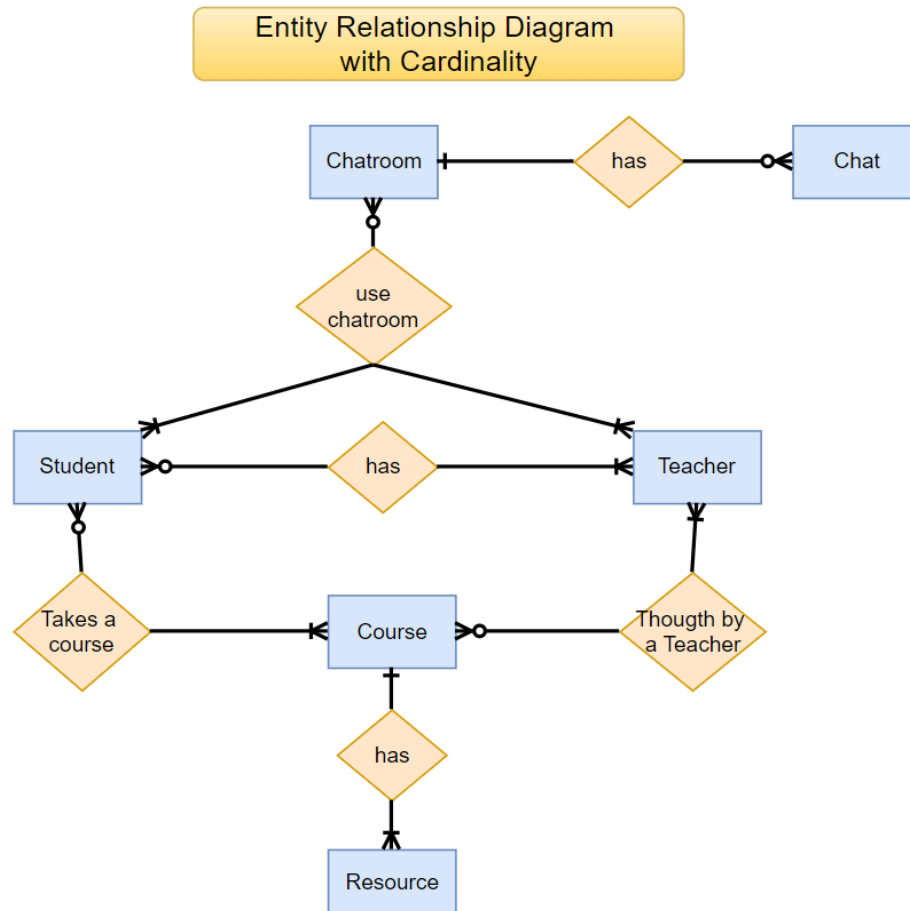
Data

For this project, the data required consisted of students, teachers and courses. To find a data set that would meet all the requirements for the project was not viable. For that reason, and due to no constraints in the data provenance, I decided to construct a data set that contains all the attributes for each table.

When constructing the data set one of the challenges was to find the relationship between entities and connect them through foreign keys. Therefore, I started designing the Entity Relationship model and creating tables that will contain data for each model.

Entity Relation Model

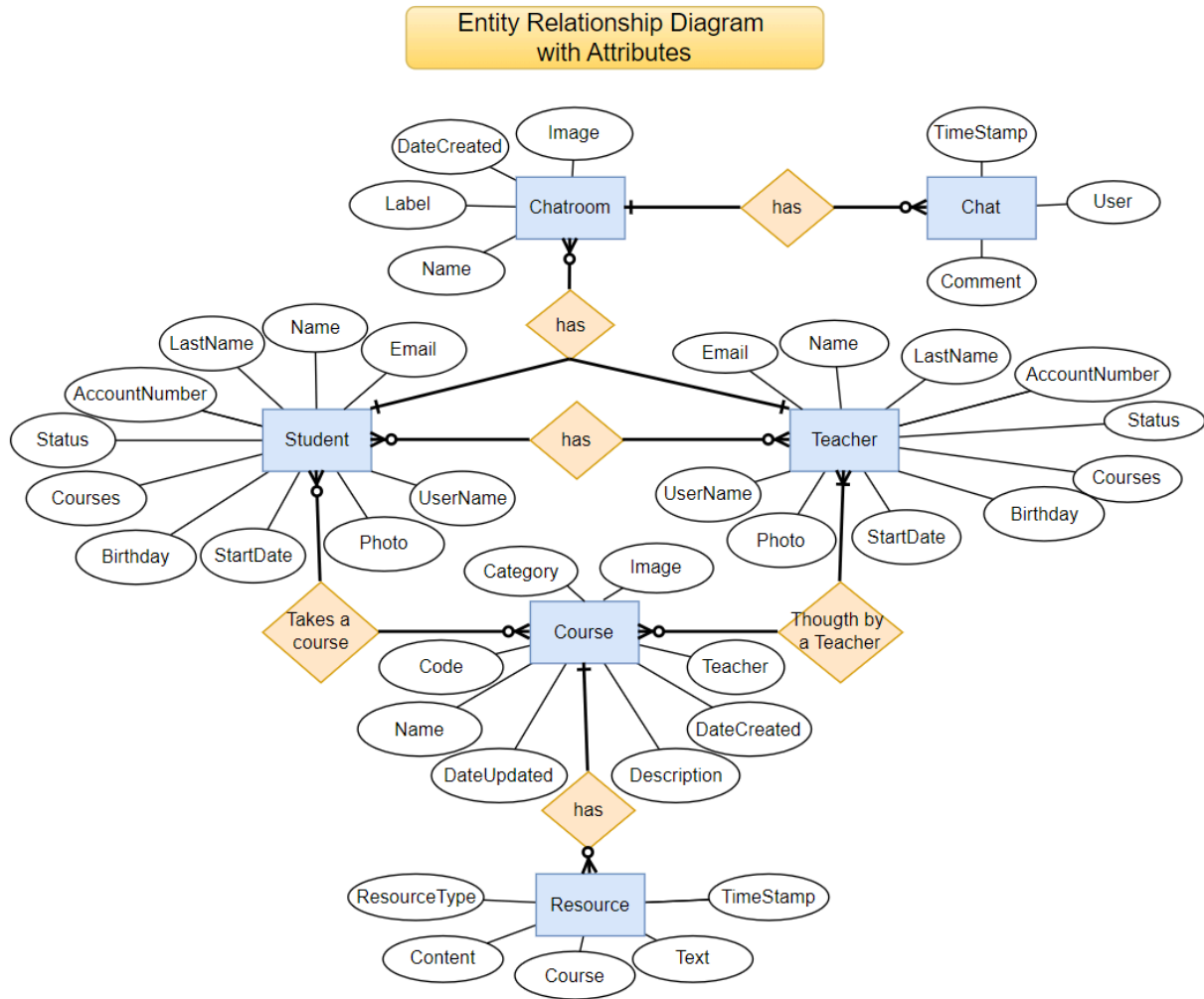
The following Diagram represents the initial data Entity Relationship model for the database used for the application.



The Student and Teacher entities show a direct relationship with the Course and with the Chat entity. The diagram shows the cardinality between entities, however it was changed to adapt for the following points:

- The creation of students without a course or a teacher
- The creation of a course without resources.

The next diagram shows the attributes used to create the models for the application database. The student and teacher entities are very similar, however the teacher entity is used as the creator of the course (as a foreign key in course).



Data Normalization

After analyzing the previous model, I could see that the data contains many-to-many relationships, multivalued attributes and values dependent on other values. Therefore the following data normalization was applied

First Normal Form

For the first normal form to be met, the models should:

- **Have unique rows:** in Django a unique primary key is added by default to all entries.
- **Have scalar values in each attribute:** the **status** attributes in student and in teacher entities contain multivalued (list of values) cells. Therefore, each status attribute was moved to its own table **StudentStatus** and **TeacherStatus**

Second Normal Form

To be in the second form, the model should:

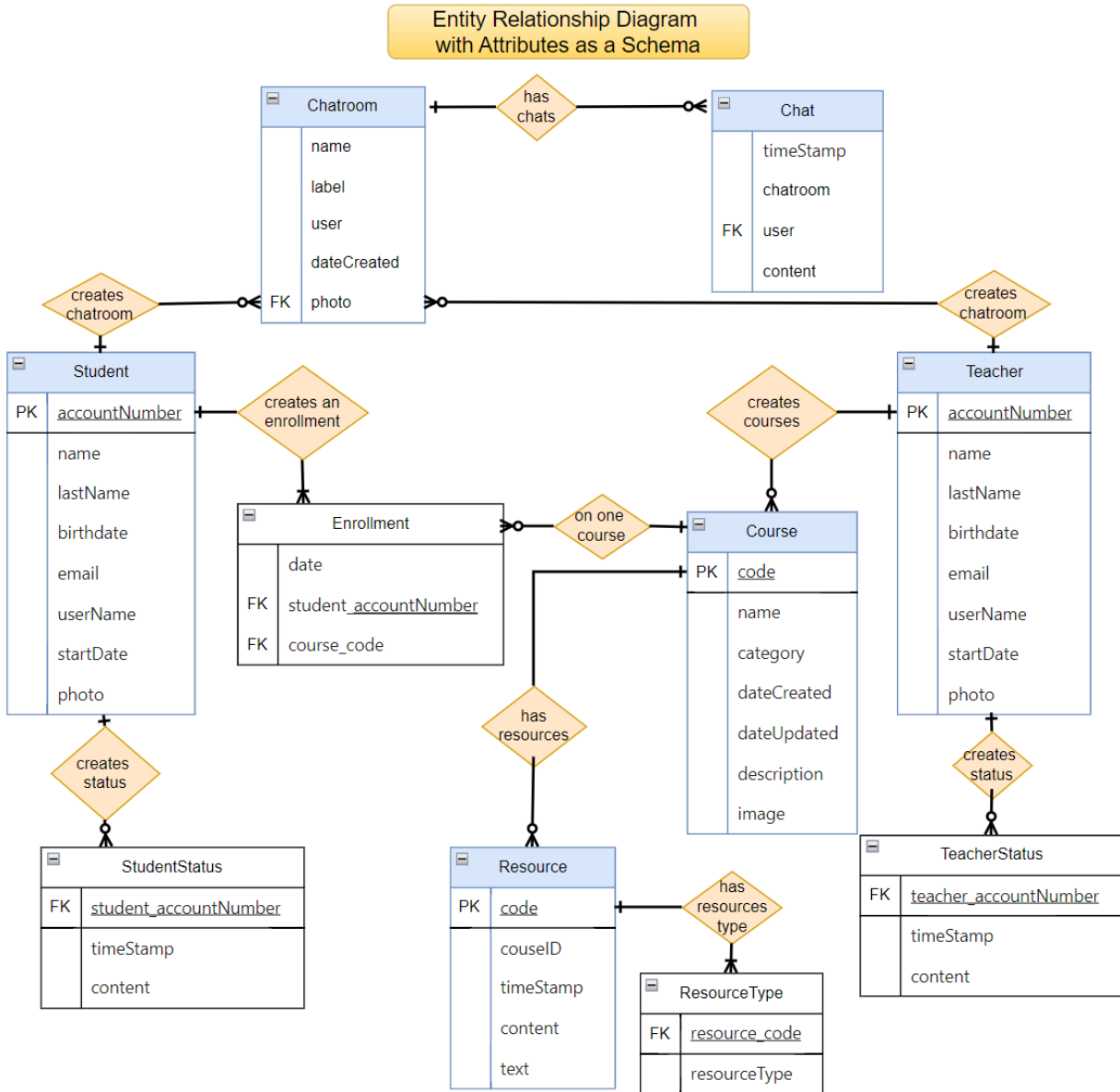
- **Meet the First Normal Form**
- **The non-prime attributes should not be dependent on other attributes dependent on the primary key[Jain]:** the students table contains an attribute with the course where the student is subscribed, therefore the student information should be repeated for each enrolled course. Therefore, a **Enrollment** table is added to separate from a Students table with unique entries of students and a Course table with unique entries of courses. The enrollment table contains only the students and courses as foreign keys.

Third Normal Form

To meet the third normal form, the model should:

- **Meet the Second Normal Form**
- **“No non-prime attribute of R is transitively dependent on the primary key” [“Third normal form - Wikipedia”]:** the data tables do not show any transitive dependencies.

After making the previous normalization and meeting the first 3 normal forms, the ER diagram was updated as follow:



Django Data Models

Django models were created for each app in the project, this helped to keep separation of concerns.

In the management app, the Django User model was customized by adding the boolean fields `is_student` and `is_teacher`. This User model was the base for the Student and Teacher models.

For the models in the applications, the attributes' types were:

- CharField for names and codes
- TextFields for descriptions
- FileField for the resources files that could accept video, pdf and text files, as well.
- ImageField for the images of courses, students and teachers

- DateTimeField for start dates and update dates
- BooleanField for the user for distinguishing from student and teacher

Migrations

Once the models were created the migration were done with the following commands:

- Python manage.py makemigrations “create database migration files based on the models” [Jain].
- Python manage.py migrate “to apply the changes to the database” [Jain]

Django Admin Setup

To access the data models and entries, the Django admin interface site was setup by creating a superuser with the following credentials:

- Username: **mayra**
- Email: mayra@email.com
- Password: **longestpass098**

Data Loading

For loading the data the following command was used:

```
python manage.py runscript load_data
```

Views and Templates

Views

To display the data in the webpage, Django views are “Python functions that take a web request and return a web response” [“Writing views”]. The response can be represented by using HTML templating. Django generic views can be very helpful to avoid repeating code.

For the eIntellectual app, the views and templating are located with their respective app and models. This helps to only access models from the views that show their data.

- Most of the views use functions: this helped to customize the view when more than one model was used.
- TemplateView is one of the simplest class views that will display any template, for example the index page and the private profile of the current user.
- ListView was used for the list that do not need to much customization and only extract and presents a list of courses, students or enrollments

- Detail view was used when only the details of a particular instance needs to be displayed: user home or course.
- DeleteView was very useful to delete one instance of an object without having to extract the object. It was used for deleting: a course, an account
- UpdateView is another view that performs CRUD operations by updating an existing view.

Templates

For templating, I use the management app to handle the home page. However, for the base to be accessed from all apps, I created a templates directory in the project folder. From there each app could access the base file directly.

The base shows the area between navigation and footer. However, it only contains information for the left and right panels, this leaves the middle column for every page content. Due to all pages containing this base content, the variables used in the base need to be passed to every view as context using a function that was called in each view.

When passing the context data to HTML templates, most of the data handling and processing is done in the views to avoid passing unnecessary data and causing data leaks.

URLs Configuration

After the initial project URL, each app URLs needs to be configured to call each view.

When showing a list of items, the URL name is directly connected to the base URL “/” or app URL:

- /courses
- /students/enrollments
- /teachers/students
- /teacher/classroom

When showing the description of a particular object and has a public view, a primary key is passed as URL parameter:

- /student/home/<int:pk>
- /teacher/home/<int:pk>
- /course/<int:pk>
- /course/<int:pk>delete
- /course/<int:pk>enroll

For the action like login and register, the URL uses the action directly connected to the user type, for example:

- student/register
- teacher/profile

- student/logout
- teacher/login
- /course/create

Validating and Securing Data

Forms

The eIntellectual app contains some forms that need to be filled up by users:

- Create a course, resources
- Enroll
- Register

These forms can be coded using HTML, however information needs to be displayed and new vulnerabilities arise if the validation process is not done correctly. For this reason Django forms were used to gather the user data. Django Forms are a powerful tool that can simplify those tasks by preparing data based on the app's models, render data by using pre-designed HTML forms, and validate and preprocess incoming data ["Working with forms"].

Validations

For previous forms I use The Django ModelForm to save the clean data, directly, once the `is_valid()` method is true. The `is_valid()` method is part of the Django ModelForm.

Security

The security mechanism used are:

- Cross Site Request Forgery (CSRF) token that "it generates a token and ensures that any requests coming back in are cross-checked against this token" [Jandhyala].
 - CSRF tokens are used in the HTML templates when displaying a Django ModelForm. I chose the CSRF token in the HTML form instead of using the decorator method because the token is applied to the whole form and not to a particular view where part of the form can come from a different view and may not be protected.

Authentication and Permissions

Users Accounts

When creating the users it was important to have separate users: students and teachers. However, having separated users makes the app more complicated when creating the chat or when giving permissions. For that reason, I created the student and teacher profiles as an

extension of Django User. Django User “ handles user accounts, groups, permissions and cookie-based user sessions” [“User authentication in Django”]

To connect the student and teacher models to the User model, it was necessary to customize the User model by adding an identifier for each type of user: `is_student` and `is_teacher`.

By using Django User models I could make use of their authentication and authorization methods, therefore creating a cleaner and secure model.

Authentication in Templating

For user authentication, Django User provides an easier to access User model that can be used in the HTML template by using the `is_authenticated` method. Once the user is authenticated, the corresponding permissions are given based on the customized attributes `is_teacher` or `is_student`.

Then the template will display content for the student, for the teacher or for the unauthorized visitors.

- Students dropdown menu
- Teacher dropdown menu
- No dropdown menu

Authentication and Permissions in Views

For the course details only one view and one template was created. The goal was to filter the information depending on the type of user. Once the user was logged in, the view would send context information to the template depending on the user's `is_student` and `is_teacher` attributes. This allowed for some content to also be displayed for anonymous users before registering or logging.

Authentication and Permissions with Decorators

The decorators are used to authenticate and to give permissions to users when visiting new pages. Decorators are Django tools that can facilitate the authentication and permissions job. The `@login_required` decorator provides a way to redirect the user to the login page if a user tries to access a page that is protected by this decorator.

The following pages in the eIntellectual project require the user to sign in to be able to access their content:

- Profiles
- Enrollments
- Teacher's students
- Created course

Testing

Unit Testing

Course models unit testing

1	__str__ function representation
2	Course.date_created happens before date.now()
3	Course.date_updated happens before date.now()
4	Course.date_updated happens after or on the date_created
5	Course.code is required
6	Course.name is required
7	Course.teacher relationship
8	Resource.code is required
9	Resource.course relationship
10	Resource.date_created happens before date.now()
11	ResourceType.resource relationship
12	Enrollment.student Enrollment.course relationship

The previous table is an example of some of the tests performed for the course model. However, more tests were implemented for the student and teacher apps. The correct way of testing is to verify that each part and function returns the correct result, displays the correct page or queries the correct data. However, due to time constraints the tests were focused on the models and the views.

App Requirements Specification Evaluation

After finishing the application the requirements are evaluated for completion

- Users
 - Users can register as students or as teachers
 - Users are able to create accounts that connect to a django User that verify the correct creation of the account
 - Users can login and logout

- Using the each student or teacher account students login using Django User
 - Users have a private profile
 - Only each respective can see their profile, however they have a Homepage that is public
 - Users can update private profile and change their own password
 - User can update some of the fields of the profile and change passwords
 - Students can delete their accounts
 - Students and teachers have a public home page to show some of the profile information.
 - The homepage contains the status section
- Teachers
 - The home page show three most recents created course
 - See a page with a list of created courses and select and edit course
 - Teacher should be able to create a new course
 - Teacher can update their own course by uploading new resources, including files
 - Teachers see a list of students enrolled in the teacher's courses
 - Teacher are able to unenroll (remove) students from the teacher's courses
- Students
 - Students home page should show three most recents enrolled courses
 - Students should see a page with a list of enrolled courses and select each one to see materials and resources
 - Students enroll themselves in a course
 - Can crate status
- Course
 - Shows a list of courses in the app
 - Display the minimum information of a course if student not enrolled or teacher is not the owner
 - Display course resources with a link to a resource when the student is enrolled or when the teacher is the owner.
 - Presents a form to create a course
 - Courses can be removed or deleted

Conclusion

After finishing the project I could see that one of the challenging parts was to understand the relationship between part of the apps and the application of the tools available to solve challenges in the project. However, I learn from each implementation:

- Applying permissions to different users. That at the same time are connected as users.
- Using the user to verify authentication to display content in the templates.
- Using Django view to avoid repeating code.

One of the parts that I would like to improve is to be able to create a functional chat app that can connect students and teachers to communicate by creating chat rooms that are categorized.

Another part that the application could benefit from, is the look of the website by using libraries that can facilitate the handling of the CSS styling.

References

Works Cited

Hellmann, Doug. "virtualenvwrapper · PyPI." *PyPI*, <https://pypi.org/project/virtualenvwrapper/>.

Accessed 29 August 2024.

"How to Write a Software Requirements Specification." *Brocoders*, 5 October 2022,

<https://medium.com/brocoders-team/how-to-write-a-software-requirements-specification-srs-document-36cefdcb09f1>. Accessed 28 August 2024.

Jain, Sandeep. "Django Basic App Model - Makemigrations and Migrate." *GeeksforGeeks*, 28 May 2024,

<https://www.geeksforgeeks.org/django-basic-app-model-makemigrations-and-migrate/>. Accessed 30 August 2024.

Jain, Sandeep. "Second Normal Form (2NF)." *GeeksforGeeks*, 22 May 2024,

<https://www.geeksforgeeks.org/second-normal-form-2nf/>. Accessed 30 August 2024.

Jandhyala, Gayatri. "What is CSRF token in Django?" *Tutorialspoint*, 2 September 2022,

<https://www.tutorialspoint.com/what-is-csrf-token-in-django>. Accessed 3 September 2024.

"Site map." *Site map - Wikipedia*, https://en.wikipedia.org/wiki/Site_map. Accessed 29 August 2024.

"Third normal form - Wikipedia." *Wikipedia, the free encyclopedia*,

https://en.wikipedia.org/wiki/Third_normal_form. Accessed 30 August 2024.

"User authentication in Django." *Django documentation*,

<https://docs.djangoproject.com/en/5.1/topics/auth/>. Accessed 3 September 2024.

"virtualenv." *virtualenv*, pypa.io, <https://virtualenv.pypa.io/en/latest/>. Accessed 29 August 2024.

“What is Windows Subsystem for Linux | Microsoft Learn.” *Learn Microsoft*, 28 November 2023,
<https://learn.microsoft.com/en-us/windows/wsl/about>. Accessed 29 August 2024.

“Working with forms.” *Django documentation*,
<https://docs.djangoproject.com/en/5.1/topics/forms/>. Accessed 31 August 2024.

“Writing views.” *Django documentation*, <https://docs.djangoproject.com/en/5.1/topics/http/views/>.
Accessed 30 August 2024.

Appendix